Three to five paragraphs of your take on ACID versus BASE architectures, past, present, and/or future

ACID and BASE approaches to database architecture have different use cases depending on an organization's needs. ACID, an acronym which stands for atomicity, consistency, isolation, and durability, thrives under relational database management systems like SQL. While BASE, meaning basically, available, soft state, and eventual consistency, has its place in non-relational database management systems such as NoSQL.

ACID systems can autoincrement or serialize primary key ids. They work well when the data is structured, normalized, not replicated, and referenced accurately through row indexing. ACID can be used for transactions, contain unique constraints, and can be vertically replicated for scaling. Scaling tends to be expensive as more disk drives, disk arrays, processors, and memory are required.

BASE systems have an assigned GUID for their primary keys. They use data that is unstructured or replicated. Data can be imported as documents in the form of JSONB files and indexed through columns rather than rows. BASE indexes follow data shards, contain no locks, and their schemas are (*on read) rather than (*on write). BASEs cannot use JOINs, aggregation commands except for COUNT, nor ALTER commands. They can be scaled inexpensively through horizontal replication with fast networks, small memory CPUS, and lots of disks.

More recently, ACID and BASE vendors have incorporated methods in their systems to allow developers to use both types of systems. Depending on the size and needs of the organization, single database systems can still be a viable option. The advent of multiple systems or hybrid systems provides a multitude of possibilities for data management.